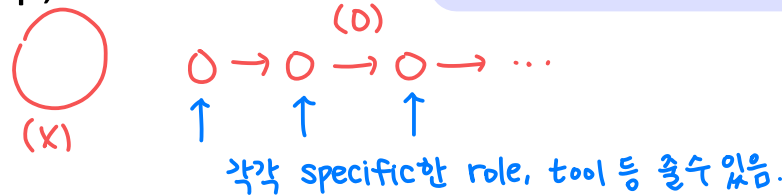

Chapter 1

Prompt Chaining

Overview (1/3)

■ Prompt chaining

- Rather than expecting an LLM to solve a complex problem in a single, monolithic step, it advocates for a **divide-and-conquer** strategy

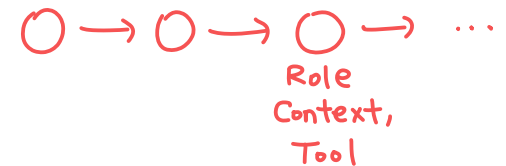


■ The core idea

- Break down the original, daunting problem into a sequence of smaller, more manageable **sub-problems**
- Each sub-problem is addressed individually through a specifically designed prompt
- The output generated from one prompt is strategically fed as input into the subsequent prompt in the chain

Overview (2/3)

- By **decomposing** a complex task, it becomes easier to understand and debug each individual step
 - Each step in the chain can **focus** on a specific aspect of the larger problem, leading to more accurate and focused outputs
- The output of one step acting as the input for the next is crucial
 - This passing of information establishes a **dependency chain**
 - The results of previous operations guide the subsequent processing
- This allows the LLM to build on its previous work, **refine** its understanding, and progressively move closer to the desired solution



Overview (3/3)

- Furthermore, it is ***not*** just about breaking down problems
 - It also enables the integration of ***external knowledge*** and ***tools***
 - At each step, the LLM can be instructed to interact with external systems, APIs, or databases, enriching its knowledge and abilities beyond its internal training data
 - This capability dramatically expands the potential of LLMs, allowing them to function not just as isolated models but as integral components of broader, more intelligent systems
- By strategically structuring the ***sequence*** of prompts, an agent can engage in tasks requiring multi-step reasoning, planning, and decision-making

Limitations of Single Prompts

- A single, complex prompt can be *inefficient* for complex tasks
 - **Instruction neglect**: parts of the prompt are overlooked
 - **Contextual drift**: the model loses track of the initial context
 - **Error propagation**: early errors amplify
 - **A longer context window**: the model gets insufficient information
 - **Hallucination**: the load increases the chance of incorrect information
- **Example**
 - Consider a query asking to **analyze** a market research report, **summarize** findings, **identify** trends with data points, and **draft** an email
 - It risks failure as the model might summarize well but fail to extract data or draft an email properly

Sequential Decomposition for Reliability

- We *break* the complex task into a focused, sequential workflow
 - Which significantly improves reliability and control
- Example (cont'd)
 1. **Initial Prompt (Summarization):** "Summarize the key findings of the following market research report: [text]."
 2. **Second Prompt (Trend Identification):** "Using the summary, identify the top three emerging trends and extract the specific data points that support each trend: [output from step 1]."
 3. **Third Prompt (Email Composition):** "Draft a concise email to the marketing team that outlines the following trends and their supporting data: [output from step 2]."

Sequential Decomposition for Reliability

- This *decomposition* allows for more granular control over the process
 - Each step is simpler and less ambiguous
 - It reduces the cognitive load on the model
 - It leads to a more accurate and reliable final output
- To ensure an accurate response for each specific task, the model can be assigned a *distinct role* at every stage
 - The initial prompt → “Market Analyst” role
 - The second prompt → “Trade Analyst” role
 - The third prompt → “Expert Documentation Writer” role
 - And so forth

The Role of Structured Output

- The reliability of a prompt chain is highly dependent on the *integrity* of the data passed between steps
 - If the output of one prompt is ambiguous or poorly formatted, the subsequent prompt may fail due to faulty input
- To mitigate this, specifying a *structured* output format, such as JSON or XML, is crucial
 - This structured format ensures that the data is machine-readable and can be precisely parsed and inserted into the next prompt without ambiguity
 - This practice minimizes errors that can arise from interpreting natural language and is a key component in building robust, multi-step LLM-based systems

Example: Structured Output

- The output from the trend identification step as a JSON object

```
{
  "trends": [
    {
      "trend_name": "AI-Powered Personalization",
      "supporting_data": "73% of consumers prefer to do business with \
                          brands that use personal information to make \
                          their shopping experiences more relevant."
    },
    {
      "trend_name": "Sustainable and Ethical Brands",
      "supporting_data": "Sales of products with ESG-related claims grew \
                          28% over the last five years, compared to 20% \
                          for products without."
    }
  ]
}
```

Practical Applications & Use Cases (1/6)

1. Information processing workflows

- Processing raw information through **multiple** transformations
- For instance, summarizing a document, extracting key entities, and then using those entities to query a database or generate a report
- A prompt chain could look like:

- Prompt 1: Extract text content from a given URL or document 특정 URL이나 문서에서 텍스트 추출
- Prompt 2: Summarize the cleaned text 텍스트 요약
- Prompt 3: Extract specific entities (e.g., names, dates, locations) from the summary or original text 이름, 날짜, 장소 등 Entity 추출
- Prompt 4: Use the entities to search an internal knowledge base 추출된 엔티티명을 이용해 내부 지식 베이스 검색
- Prompt 5: Generate a final report incorporating the summary, entities, and search results 요약, 엔티티명, 검색 결과를 통합하여 최종 보고서 생성

↪ 요즘 개발자는 이거 잘하는 사람

Practical Applications & Use Cases (2/6)

2. Complex query answering

- Answering complex questions that require **multiple** steps of reasoning or information retrieval is a prime use case
- For example, "What were the main causes of the stock market crash in 1929, and how did government policy respond?"

- Prompt 1: Identify the core sub-questions in the user's query (causes of crash, government response) 질문을 '붕괴 원인'과 '정부 대응'이라는 핵심 하위 질문으로 분리
- Prompt 2: Research or retrieve information specifically about the causes of the 1929 crash 붕괴 원인에 대해 구체적으로 정보 수집
- Prompt 3: Research or retrieve information specifically about the government's policy response to the 1929 stock market crash 정부의 정책 대응에 대해 구체적으로 정보 수집
- Prompt 4: Synthesize the information from steps 2 and 3 into a coherent answer to the original query 수집된 두 정보를 종합하여 하나의 일관된 답변으로 작성

Practical Applications & Use Cases (3/6)

3. Data extraction and transformation

- The conversion of unstructured text into a structured format is typically achieved through an **iterative** process
- It requires sequential modifications to improve the accuracy and completeness of the output

- Prompt 1: Attempt to extract specific fields (e.g., name, address, amount) from an invoice document 송장에서 이름, 주소, 금액 등의 특정 필드 추출 시도
- Processing: Check if all required fields were extracted and if they meet ^{처리} format requirements 모든 필드가 정확히 추출되었는지, 형식이 맞는지 검증
- Prompt 2 (Conditional): If fields are missing or malformed, craft a new ^{조건부} prompt asking the model to specifically find the missing/malformed ^{필드가 누락되거나 오류가 있다면, 이전의 실패 문맥을 제공하며 다시 찾도록 요청} information, perhaps providing context from the failed attempt
- Processing: Validate the results again. Repeat if necessary ^{이 과정을 반복하며 최종적으로 검증된 데이터를 출력}
- Output: Provide the extracted, validated structured data

Practical Applications & Use Cases (4/6)

4. Content generation workflows

- Composing complex content is typically **decomposed** into distinct phases
- It may include initial ideation, structural outlining, drafting, and subsequent revision

idea를 낸 LLM과 평가하는 LLM은 서로 다른걸 쓰는게 좋음.

- Prompt 1: Generate 5 topic ideas based on a user's general interest 주제 제안
- Processing: Allow the user to select one idea or automatically choose the best one 아이디어 선택
- Prompt 2: Based on the selected topic, generate a detailed outline 세부 개요 생성
- Prompt 3: Write a draft section based on the first point in the outline
- Prompt 4: Write a draft section based on the second point in the outline, providing the previous section for context. Continue this for all outline points
개요별 초안 작성
(이전 개요 맥락 반영)
- Prompt 5: Review and refine the complete draft for coherence, tone, and grammar 일관성과 어조 등을 고려한 최종 검토 및 정제

Practical Applications & Use Cases (5/6)

5. Code generation and refinement

- The generation of functional code is typically a **multi-stage** process
- It requires a problem to be decomposed into a sequence of discrete logical operations that are executed progressively

- Prompt 1: Understand the user's request for a code function. Generate pseudocode or an outline 사용자 요청을 이해하고 의사코드나 개요를 먼저 생성
- Prompt 2: Write the initial code draft based on the outline 개요를 바탕으로 초안 코드 작성
- Prompt 3: Identify potential errors or areas for improvement in the code (perhaps using a static analysis tool or another LLM call) 네가 네 코드를 비판해봐
정적 분석 도구나 다른 LLM 호출을 통해 코드의 잠재적 오류나 개선점 식별
- Prompt 4: Rewrite or refine the code based on the identified issues 식별된 문제를 바탕으로 코드를 재작성하거나 정제
- Prompt 5: Add documentation or test cases 문서화나 테스트 케이스 추가
ex) README.md

Practical Applications & Use Cases (6/6)

6. Multimodal and multi-step reasoning

- Analyzing datasets with diverse modalities necessitates **breaking down** the problem into smaller, prompt-based tasks
- For example, interpreting an image that contains a picture with embedded text, labels highlighting specific text segments, and tabular data explaining each label, requires such an approach.

- Prompt 1: ^{이미지에서 텍스트 추출 및 이해} Extract and comprehend the text from the user's image request
- Prompt 2: Link the extracted image text with its corresponding labels
- Prompt 3: ^{추출된 텍스트와 label 연결} Interpret the gathered information using a table to determine the required output ^{표 정보를 해석하여 필요한 최종 결과 도출}

Hands-On Code Example

- Install the required libraries first

```
pip install langchain langchain-community langchain-openai langgraph
```

- Example: a **two-step** prompt chain of a data processing pipeline

코드 검증, 이해는 가능해야 함.

```
import os
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# For better security, load environment variables from a .env file
# from dotenv import load_dotenv

# load_dotenv() 환경변수 불러오기
# Make sure your OPENAI_API_KEY is set in the .env file
```



```
# Initialize the Language Model
llm = ChatOpenAI(temperature=0)
```

LLM 선택



```
# --- Prompt 1: Extract Information ---
prompt_extract = ChatPromptTemplate.from_template(
    "Extract the technical specifications from the following text:\n\n{text_input}"
)
```

text_input이 아직 비어있으니 일단 템플릿만
text_input에서 기술 사양을 추출하는 프롬프트

```
# --- Prompt 2: Transform to JSON ---
prompt_transform = ChatPromptTemplate.from_template(
    "Transform the following specifications into a JSON object with 'cpu', 'memory',  
and 'storage' as keys:\n\n{specifications}"
)
```

사양(specifications)을 cpu, memory, storage 키를 가진 JSON 객체로 변환하는 프롬프트

```
# --- Build the Chain using LCEL ---
# The StrOutputParser() converts the LLM's message output to a simple string.
extraction_chain = prompt_extract | llm | StrOutputParser()
```

```
# The full chain passes the output of the extraction chain into the 'specifications'  
# variable for the transformation prompt.
```

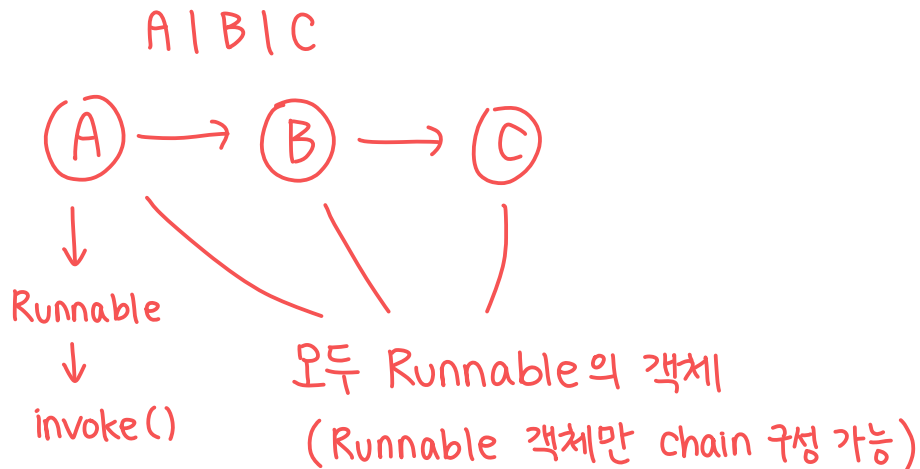
```
full_chain = (
    ({ "specifications": extraction_chain })
    | prompt_transform
    | llm
    | StrOutputParser()
)
```

String type
AI Message
String으로 변환
Json
String

```
# --- Run the Chain ---
input_text = "The new laptop model features a 3.5 GHz octa-core processor, 16GB of
RAM, and a 1TB NVMe SSD."

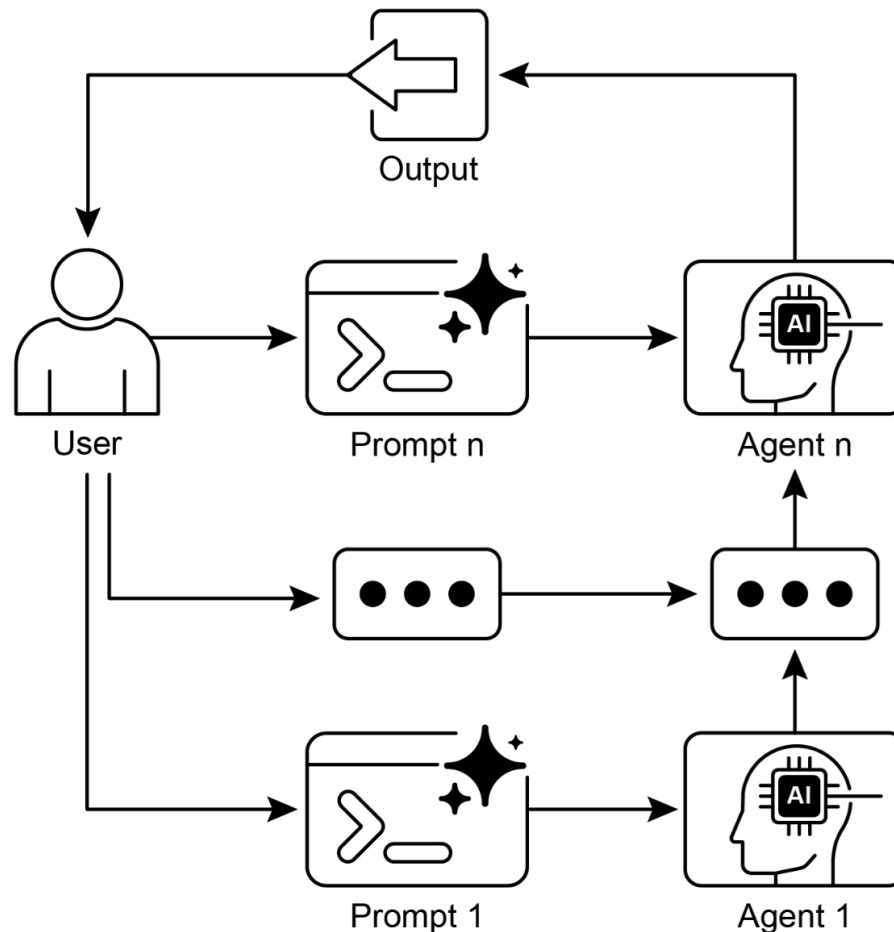
# Execute the chain with the input text dictionary.
final_result = full_chain.invoke({"text_input": input_text})

print("\n--- Final JSON Output ---")
print(final_result)
```



Visual Summary: Prompt Chaining Pattern

- Agents receive a *series* of prompts from the user
 - With the output of each agent serving as the input for the next in the chain

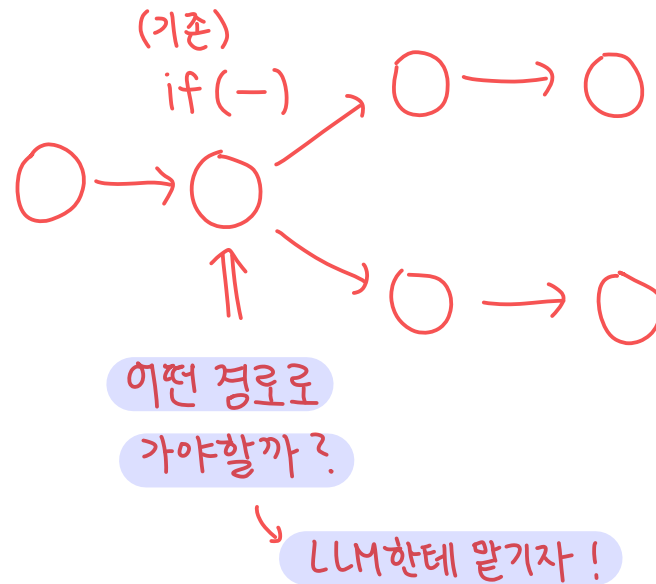


Key Takeaways

- Prompt Chaining breaks down complex tasks into a ***sequence*** of smaller, focused steps (a.k.a. the Pipeline pattern)
- Each step in a chain involves an LLM call or processing logic, ***using*** the output of the previous step as input
- This pattern improves the ***reliability*** and ***manageability*** of complex interactions with language models
- Frameworks like LangChain, LangGraph, and Google ADK provide robust tools to define, manage, and execute these ***multi-step*** sequences

Chapter 2

Routing



Overview (1/2)

- The applicability of prompt chaining is limited in scenarios requiring ***adaptive*** responses
 - Because it is a technique for executing deterministic, linear workflows
- Real-world agentic systems must often ***arbitrate*** between multiple potential actions based on contingent factors
 - E.g., the state of the environment, user input, the outcome of a preceding operation
- This capacity for dynamic control of flow is achieved through a mechanism known as ***routing***
 - To different specialized functions, tools, or sub-processes

Overview (2/2)

- Routing introduces *conditional* logic into agent's operations
 - Enables a shift from a fixed execution path
 - The agent dynamically evaluates specific criteria to *select* from a set of possible subsequent actions
 - This allows for more flexible and context-aware system behavior
- Routing enables a system to move *beyond* deterministic sequential processing
 - It facilitates the development of more adaptive execution flows
 - It can respond dynamically and appropriately to a wider range of inputs and state changes

Example: An Agent for Customer Inquiries

Chatbot

1. Analyze the user's query

2. Route the query based on its intent: intent는 LLM이 알아냄.

- If the intent is "check order status"
 - Route to a sub-agent or tool chain that interacts with the order database
- If the intent is "product information"
 - Route to a sub-agent or chain that searches the product catalog
- If the intent is "technical support"
 - Route to a different chain that accesses troubleshooting guides or escalates to a human
- If the intent is unclear 사람이 개입
 - Route to a clarification sub-agent or prompt chain

Routing Mechanisms (1/2)

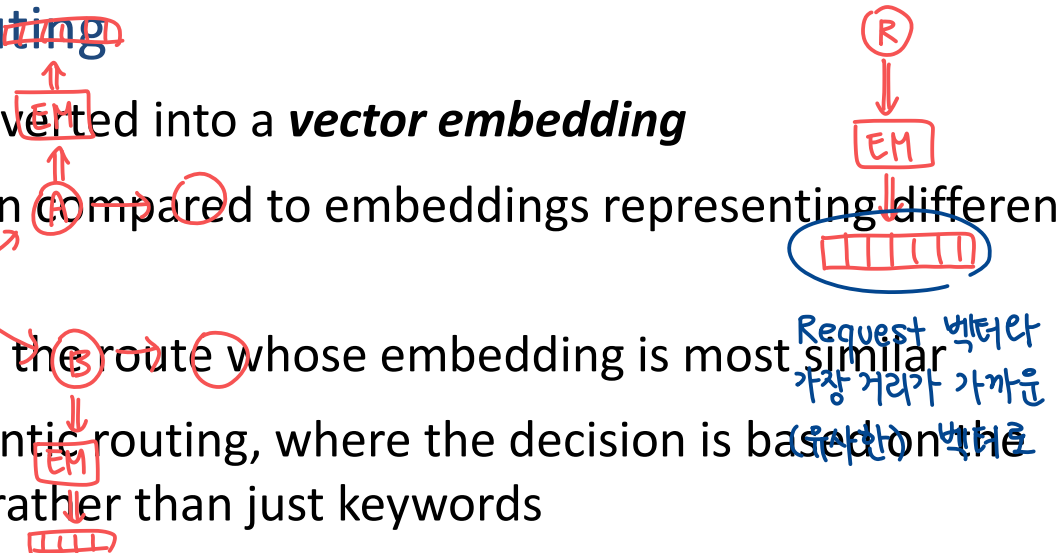
■ LLM-based routing

- The language model **itself** can be prompted to analyze the input and output a specific identifier or instruction that indicates the next step
- Example: "Analyze the following user query and output only the category: 'Order Status', 'Product Info', 'Technical Support', or 'Other'."

단점 : LLM이 의도를 잘못 분류했을 경우 컨트롤 불가 → 프롬프트를 refine 해줘야 됨.

■ Embedding-based routing

- The input query is converted into a **vector embedding**
- This embedding is then compared to embeddings representing different routes or capabilities
- The query is routed to the route whose embedding is most similar
- This is useful for semantic routing, where the decision is based on the meaning of the input rather than just keywords



Routing Mechanisms (2/2)

- Rule-based routing *if 문 (옛날 방식)*
 - This involves using **predefined** rules or logic (e.g., if-else statements, switch cases) based on keywords, patterns, or structured data extracted from the input
 - This can be faster and more deterministic than LLM-based routing
 - But it is less flexible for handling nuanced or novel inputs
- Machine learning model-based routing
 - It employs a **discriminative model**, such as a classifier, that has been specifically trained on a small set of labeled data to perform a routing task *단점 : 왜 그쪽으로 routing 하는지는 블랙박스임. + 훈련 비용 ↑*
 - Its key characteristic is the supervised fine-tuning process
 - The routing logic is encoded within the model's learned weights

Practical Applications & Use Cases (1/3)

1. Human-computer interaction

- Such as with virtual assistants or AI-driven tutors
- Routing is employed to interpret user *intent*
- An initial analysis of a natural language query determines the most appropriate subsequent action
 - Is it invoking a specific information retrieval tool?
 - Is it escalating to a human operator?
 - Is it selecting the next module in a curriculum based on user performance?
- This allows the system to move beyond linear dialogue flows and respond contextually

Practical Applications & Use Cases (2/3)

2. Automated data and document processing pipelines

- Routing serves as a classification and distribution function
- Incoming data, such as emails, support tickets, or API payloads, is ***analyzed*** based on content, metadata, or format
- The system then directs each item to a corresponding workflow
 - A sales lead ingestion process
 - A specific data transformation function for JSON or CSV formats
 - An urgent issue escalation path

Practical Applications & Use Cases (3/3)

3. Complex systems involving multiple specialized tools or agents

- Routing acts as a high-level *dispatcher*
- A research system assigns task to the most suitable agent based on the current objective
 - “Search” agent
 - “Summarization” agent
 - “Analysis” agent
- Similarly, an AI coding assistant identifies the programming language and user's intent before passing a code snippet to the correct specialized tool
 - “Debugging” agent
 - “Explanation” agent
 - “Translation” agent

Hands-On Code Example (LangChain)

- Install the necessary libraries

```
pip install langchain langgraph google-cloud-aiplatform  
langchain-google-genai google-adk deprecated pydantic
```

- Example: a simple agent-like system

```
from langchain_google_genai import ChatGoogleGenerativeAI  
from langchain_core.prompts import ChatPromptTemplate  
from langchain_core.output_parsers import StrOutputParser  
from langchain_core.runnables import RunnablePassthrough, RunnableBranch  
  
# --- Configuration ---  
try:  
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)  
    print(f"Language model initialized: {llm.model}")  
except Exception as e:  
    print(f"Error initializing language model: {e}")  
    llm = None
```

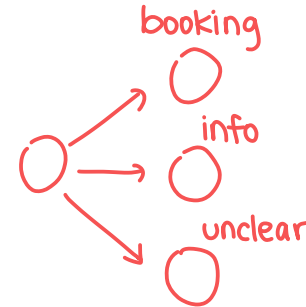
분류된 결과에 따라 실제로 동작할 함수 (Sub-Agent 역할을 하는 핸들러)

```
# --- Define Simulated Sub-Agent Handlers ---
```

```
def booking_handler(request: str) -> str:
    """Simulates the Booking Agent handling a request."""
    print("\n--- DELEGATING TO BOOKING HANDLER ---")
    return (
        f"Booking Handler processed request: '{request}'. "
        "Result: Simulated booking action."
    )

def info_handler(request: str) -> str:
    """Simulates the Info Agent handling a request."""
    print("\n--- DELEGATING TO INFO HANDLER ---")
    return (
        f"Info Handler processed request: '{request}'. "
        "Result: Simulated information retrieval."
    )

def unclear_handler(request: str) -> str:
    """Handles requests that couldn't be delegated."""
    print("\n--- HANDLING UNCLEAR REQUEST ---")
    return (
        f"Coordinator could not delegate request: '{request}'. "
        "Please clarify."
    )
```



```
# --- Define Coordinator Router Chain ---
```

```
# This chain decides which handler to delegate to.
```

```
coordinator_router_prompt = ChatPromptTemplate.from_messages([ Intent를 LLM한테 판단하도록
```

```
    ("system",
```

```
        """
```

시스템 프롬프트 내용

```
Analyze the user's request and determine which specialist handler should process it.
```

```
- If the request is related to booking flights or hotels, output 'booker'.
```

```
- For all other general information questions, output 'info'.
```

```
- If the request is unclear or doesn't fit either category, output 'unclear'.
```

```
ONLY output one word: 'booker', 'info', or 'unclear'.
```

```
        """.strip()
```

```
    ),
```

```
    ("user", "{request}")
```

```
])
```

user가 입력한
프롬프트

```
if llm:
```

```
    coordinator_router_chain = coordinator_router_prompt | llm | StrOutputParser()
```

~~booker or info or unclear~~

Define the branches for the RunnableBranch

branches = { Dictionary

"booker": RunnablePassthrough.assign(
 output=lambda x: booking_handler(x['request']['request'])

),
"info": RunnablePassthrough.assign(
 output=lambda x: info_handler(x['request']['request'])

),
"unclear": RunnablePassthrough.assign(
 output=lambda x: unclear_handler(x['request']['request'])

),
},

Create the RunnableBranch. It takes the output of the router chain

and routes the original input ('request') to the corresponding handler.

delegation_branch = RunnableBranch(
 (lambda x: x['decision'].strip() == 'booker', branches["booker"]),
 (lambda x: x['decision'].strip() == 'info', branches["info"]),
 branches["unclear"] # Default branch for 'unclear' or any other output

)

Combine the router chain and the delegation branch into a single runnable

The router chain's output ('decision') is passed along with

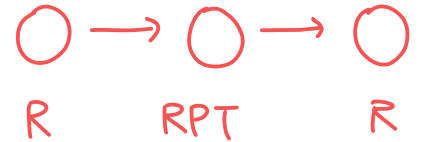
the original input ('request')

coordinator_agent = {

"decision": coordinator_router_chain, 'booker'

"request": RunnablePassthrough() {"request": ~}

} | delegation_branch | (lambda x: x['output']) # Extract the final output



Json

{ decision : 'booker'
 request : ~

⊕ output : ... }

추가!

output 값

```
# --- Example Usage ---
def main():
    if not llm:
        print("\nSkipping execution due to LLM initialization failure.")
        return

    print("--- Running with a booking request ---")
    request_a = "Book me a flight to London."
    result_a = coordinator_agent.invoke({"request": request_a})
    print(f"Final Result A: {result_a}")

    print("\n--- Running with an info request ---")
    request_b = "What is the capital of Italy?"
    result_b = coordinator_agent.invoke({"request": request_b})
    print(f"Final Result B: {result_b}")

    print("\n--- Running with an unclear request ---")
    request_c = "Tell me about quantum physics."
    result_c = coordinator_agent.invoke({"request": request_c})
    print(f"Final Result C: {result_c}")

if __name__ == "__main__":
    main()
```

Hands-On Code Example (Google ADK)

- Example: an ADK application using Google's ADK library

```
From typing import Dict, Any, Optional
from google.adk.agents import Agent
from google.adk.runners import InMemoryRunner
from google.adk.tools import FunctionTool
from google.genai import types
from google.adk.events import Event

# --- Define Tool Functions ---
# These functions simulate the actions of the specialist agents.

def booking_handler(request: str) -> str:
    """
    Handles booking requests for flights and hotels.
    Args: 이것들을 잘 써줘야 LLM이 보고 적절한 handler를 호출함.
        request: The user's request for a booking.
    Returns:
        A confirmation message that the booking was handled.
    """
    print("----- Booking Handler Called -----")
    return f"Booking action for '{request}' has been simulated."
```

```

def info_handler(request: str) -> str:
    """
    Handles general information requests.
    Args:
        request: The user's question.
    Returns:
        A message indicating the information request was handled.
    """
    print("----- Info Handler Called -----")
    return (
        f"Information request for '{request}'. "
        "Result: Simulated information retrieval."
    )

def unclear_handler(request: str) -> str:
    """Handles requests that couldn't be delegated."""
    return (
        f"Coordinator could not delegate request: '{request }'. "
        "Please clarify."
    )

# --- Create Tools from Functions ---
booking_tool = FunctionTool(booking_handler)
info_tool = FunctionTool(info_handler)

```

하이코로 만들어줌. 일반 함수

```
# Define specialized sub-agents equipped with their respective tools
booking_agent = Agent(
    name="Booker",
    model="gemini-2.0-flash",
    description="A specialized agent that handles all flight
    and hotel booking requests by calling the booking tool.",
    tools=[booking_tool]
)

info_agent = Agent(
    name="Info",
    model="gemini-2.0-flash",
    description="A specialized agent that provides general information
    and answers user questions by calling the info tool.",
    tools=[info_tool]
)
```

```

# Define the parent agent with explicit delegation instructions
coordinator = Agent(
    name="Coordinator",
    model="gemini-2.0-flash",
    instruction=(
        "You are the main coordinator. Your only task is to analyze
        incoming user requests "
        "and delegate them to the appropriate specialist agent.
        Do not try to answer the user directly.\n"
        "- For any requests related to booking flights or hotels,
        delegate to the 'Booker' agent.\n"
        "- For all other general information questions, delegate to
        the 'Info' agent."
    ),
    description="A coordinator that routes user requests to the
    correct specialist agent.",
    # The presence of sub_agents enables LLM-driven delegation by default.
    sub_agents=[booking_agent, info_agent]
)

```

```

# --- Execution Logic ---
async def run_coordinator(runner: InMemoryRunner, request: str):
    """Runs the coordinator agent with a given request and delegates."""
    print(f"\n--- Running Coordinator with request: '{request}' ---")
    final_result = ""
    try:
        user_id = "user_123"
        session_id = str(uuid.uuid4())
        await runner.session_service.create_session(
            app_name=runner.app_name, user_id=user_id, session_id=session_id
        )

        for event in runner.run(user_id=user_id, session_id=session_id,
                                new_message=types.Content(role='user', parts=[types.Part(text=request)])):
            if event.is_final_response() and event.content:
                if hasattr(event.content, 'text') and event.content.text:
                    final_result = event.content.text
                elif event.content.parts:
                    text_parts = [part.text for part in event.content.parts if part.text]
                    final_result = "".join(text_parts)
                break

        print(f"Coordinator Final Response: {final_result}")
        return final_result
    except Exception as e:
        print(f"An error occurred while processing your request: {e}")
        return f"An error occurred while processing your request: {e}"

```

```

async def main():
    """Main function to run the ADK example."""
    print("--- Google ADK Routing Example (ADK Auto-Flow Style) ---")
    print("Note: This requires Google ADK installed and authenticated.")

    runner = InMemoryRunner(coordinator)

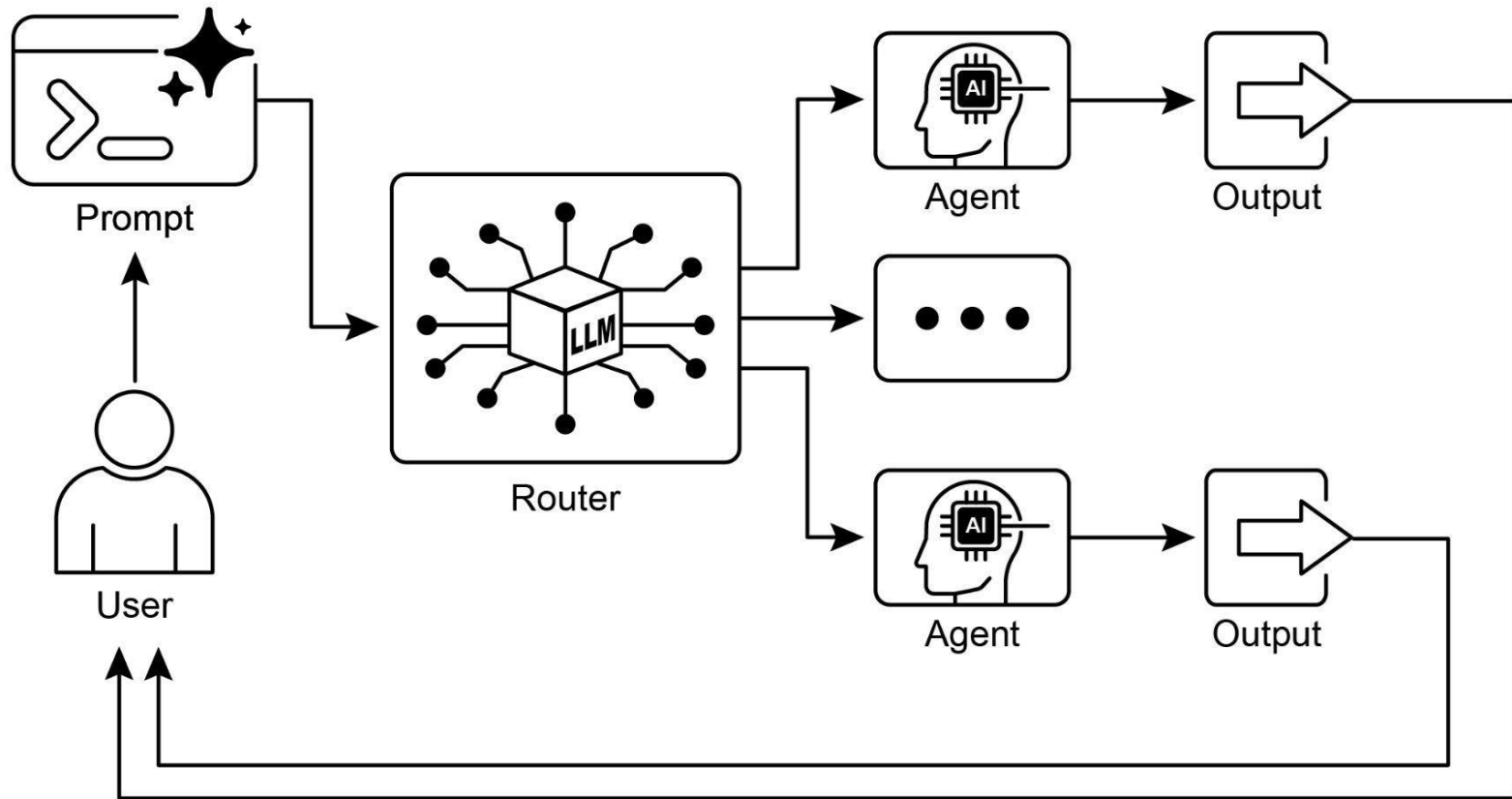
    # Example Usage
    result_a = await run_coordinator(runner, "Book me a hotel in Paris.")
    print(f"Final Output A: {result_a}")
    result_b = await run_coordinator(runner, "What is the highest mountain in the
world?")
    print(f"Final Output B: {result_b}")
    result_c = await run_coordinator(runner, "Tell me a random fact.")
    print(f"Final Output C: {result_c}")
    result_d = await run_coordinator(runner, "Find flights to Tokyo next month.")
    print(f"Final Output D: {result_d}")

if __name__ == "__main__":
    import nest_asyncio
    nest_asyncio.apply()
    await main()

```


Visual Summary: Routing Pattern

- Router pattern (using an LLM as a Router)



Key Takeaways

- Routing enables agents to make *dynamic* decisions about the next step in a workflow based on conditions
- It allows agents to handle diverse inputs and *adapt* their behavior, moving beyond linear execution
- Routing *logic* can be implemented using LLMs, rule-based systems, or embedding similarity
- Frameworks like LangGraph and Google ADK provide structured ways to define and manage *routing* within agent workflows, albeit with different architectural approaches